registered Edge Service in Keycloak as an OAuth2 Client, defining both a client identifier (`edge-service`) and a shared secret (`polar-keycloak-secret`). Now we'll use that information to tell Spring Security how to interact with Keycloak.

Open the application.yml file in your Edge Service project, and add the following configuration.

Listing 11.6 Configuring Edge Service as an OAuth2 Client

```
spring:
  security:
    oauth2:
      client:
        registration:
          keycloak:                                    ❶
            client-id: edge-service                    ❷
            client-secret: polar-keycloak-secret       ❸
            scope: openid                              ❹
        provider:
          keycloak:                                    ❺
            issuer-uri:
➥http://localhost:8080/realms/PolarBookshop           ❻
```

❶ The name identifying the Client registration in Spring Security (called "registrationId"). It can be any string.

❷ The OAuth2 Client identifier as defined in Keycloak

❸ The shared secret used by the Client to authenticate with Keycloak

❹ The list of scopes the Client would like to have access to. The openid scope triggers OIDC authentication on top of OAuth2.

❺ The same name used as "registrationId" a few lines above

❻ The Keycloak URL providing information about all the relevant OAuth2 and OIDC endpoints for the specific realm

Each Client registration in Spring Security must have an identifier (*registrationId*). In this example, it's `keycloak`. The registration identifier is used to build the URL where Spring Security receives the Authorization Code from Keycloak. The default URL template is